Introduction

Every automation engineer eventually builds the same workflow: pull a list of items, hit an API once per item, move on. It works perfectly in testing. Five items, ten items — every call succeeds, and it feels done.

Then it goes to production with five hundred items, and somewhere around item forty-seven, the API stops cooperating. Requests start coming back with a 429 status code — "Too Many Requests" — and depending on how the workflow is built, the rest of the run can collapse entirely.

This is one of the most common failure modes in low-code automation, and it's worth understanding deeply, because the underlying problem isn't specific to n8n. It shows up anywhere a program calls an external service faster than that service is willing to accept: Python scripts hitting REST APIs, serverless functions fanning out, even naive `for` loops in a backend job. The pattern you'll learn here — group work into batches, pace it with a deliberate delay, and retry failures intelligently — is a general-purpose engineering technique, not a workaround specific to one tool.

For anyone preparing for backend or infrastructure interviews, rate limiting is also a recurring topic. Interviewers want to know if you understand why unthrottled clients break shared resources, and whether you reach for the right level of complexity to fix it — not every throttling problem needs a message queue or a distributed rate limiter. Sometimes it needs exactly two nodes.

Problem Overview

Here's the situation in plain terms: you have a workflow that reads a list of items — say, 500 records — and for each one, calls an external API. There's no batching logic, no delay between calls, nothing moderating the speed. n8n (or any workflow engine, or any script) will process each item as fast as the engine can push it through.

The API, however, has a hard requirement: something like ten requests per second, enforced server-side. This limit is invisible to your workflow. Nothing in n8n inspects the API's documentation or negotiates a safe pace automatically. The engine just sends requests as quickly as it can, and the API has to defend itself the moment that rate is exceeded — which it does by rejecting calls with a 429 status code instead of queuing them politely.

The failure is deterministic once you cross the threshold. On 5 test items you probably never hit the ceiling, so everything looks fine. On 500 items, you blow past it almost immediately, and depending on

how your workflow handles failures, either a chunk of items gets dropped, or the entire execution halts.

The fix has two parts that have to work together:

1. **Group items into batches** instead of sending them one at a time, unthrottled.
2. **Pause between batches** long enough that you stay under the API's actual rate ceiling.

Neither alone solves the problem. Batching without pacing just sends bursts faster. Pacing without batching (a wait after every single item) works but is usually slower and harder to reason about than it needs to be.

Example

Imagine you're syncing 500 user records to a CRM's API, and the CRM enforces 10 requests per second.

Without batching or pacing: - Items 1–46 succeed because the burst hasn't yet exceeded the server's tolerance. - Item 47 onward: the CRM starts returning `429 Too Many Requests`. - Depending on error handling, the remaining ~453 items either fail outright or the whole execution stops.

With batching (10 items) and a 2-second wait between batches: - Batch 1 (items 1–10) fires, all succeed. - Wait node pauses 2 seconds. - Batch 2 (items 11–20) fires, all succeed. - This repeats 50 times until all 500 items are processed. - Effective rate: 10 requests every ~2 seconds, comfortably under the 10-requests-per-second ceiling. - Result: zero 429 errors, full completion.

The key output to notice here isn't just "it works" — it's that the throughput is now a number you *chose*, rather than a number the server enforced on you reactively through failures.

Intuition

The instinct many engineers have when they first see a 429 error is to reach for something heavy: exponential backoff with jitter, a message queue, a token-bucket rate limiter, maybe even a dedicated throttling service. That instinct comes from experience with systems where *many independent clients* hammer one API concurrently — a fleet of microservices, a swarm of background workers, thousands of browser clients. In that world, jitter exists specifically to stop multiple callers from retrying in lockstep and re-causing the exact spike that triggered the failure.

But look closely at what's actually happening in this workflow: it's **one execution, calling the endpoint serially**. There's no fleet. There's no concurrency to desynchronize. The entire "rate

limiting problem" reduces to a much simpler one: *don't send requests faster than the server accepts them, from this single thread of execution.*

Once you see it that way, the solution almost writes itself: process in small groups, and insert a deliberate pause between groups so the average rate stays under the ceiling. No queue, no broker, no distributed coordination — because there's nothing to coordinate between. This is the kind of simplification experienced engineers look for before reaching for infrastructure: match the solution's complexity to the actual shape of the problem, not to the scariest version of the problem you've seen before.

The remaining question is tactical: how big should each batch be, and how long should the pause be? Start conservative — well under the documented limit — and tighten it once you've watched the real behavior, ideally by reading the rate-limit headers the API sends back.

Brute Force Solution

Idea: Send every request as fast as possible, no batching, no waiting. This is the default behavior if you build the workflow naively.

Algorithm: 1. Fetch all items. 2. For each item, call the API immediately. 3. Move to the next item with no delay.

Advantages: - Simplest possible workflow — no extra nodes. - Fastest possible completion time, assuming no limit existed.

Disadvantages: - Breaks the moment item count × calls-per-second exceeds the API's real ceiling. - Failure is partial and unpredictable — succeeds on small test sets, fails in production. - No mechanism to recover from transient errors (timeouts, network blips) either.

Time complexity: O(n) calls, but with a hidden failure mode once `n` is large enough to cross the rate ceiling. **Space complexity:** O(1) beyond the item list — nothing is batched or held in memory longer than one item at a time.

This "solution" is really the bug report, not a fix — but it's worth stating explicitly because it's what nearly every first-draft integration looks like.

Optimal Solution

The fix is a three-part pattern: **batch, pace, retry.**

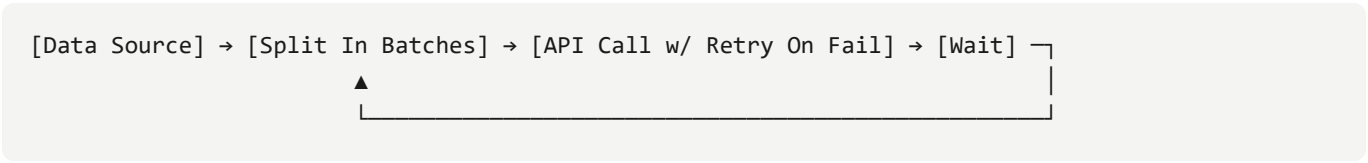
Step 1 — Batch. A `Split In Batches` node sits right after the data source. Instead of 500 items flowing individually into the next node, they're grouped — for example, into chunks of 10. Ten is a reasonable starting point because it's comfortably under most APIs' per-second limits; raise it once you've confirmed the real ceiling.

Step 2 — Pace. A `Wait` node follows the API call. Critically, its output loops *back* to the `Split In Batches` node rather than continuing straight ahead. This loop is the entire mechanism:

Without the loop	With the loop
Batch 1 processes, Wait fires once, then every remaining batch fires immediately	Batch 1 processes → wait → Batch 2 processes → wait → … until done
Still hits the rate wall	Rate stays capped the whole run

Skipping the loop is the single most common mistake in building this pattern — it looks correct on a one-batch test and only reveals the bug at scale, exactly like the original problem.

Step 3 — Retry. Batching and pacing solve the *rate limit* problem, but not reliability. A single network blip in batch 30 — unrelated to rate limiting — can still kill the entire execution if the API-call node has no error handling. Turning on `Retry On Fail` gives that one request a couple more attempts instead of taking down the run. Set the retry delay to a few seconds, not zero — retrying instantly just hits the same overloaded or glitching server again. If the API returns a `Retry-After` header, use that exact value instead of guessing.



Python Code

While this workflow is built visually in n8n, the identical pattern in Python makes the mechanics explicit — useful both for understanding and for anyone automating similar logic outside a low-code tool.

```
import time
import requests
from requests.exceptions import RequestException

BATCH_SIZE = 10
WAIT_SECONDS = 2
MAX_RETRIES = 3
RETRY_DELAY_SECONDS = 3
```

```
def call_api_with_retry(item, session, max_retries=MAX_RETRIES):
    """Call the API for one item, retrying on transient failures."""
    for attempt in range(1, max_retries + 1):
        try:
            response = session.post("https://api.example.com/sync", json=item)
            response.raise_for_status()
            return response.json()
        except RequestException:
            if attempt == max_retries:
                raise
            time.sleep(RETRY_DELAY_SECONDS)


def process_in_batches(items, batch_size=BATCH_SIZE, wait_seconds=WAIT_SECONDS):
    """Process items in rate-limit-safe batches with a pause between groups."""
    session = requests.Session()
    results = []

    for start in range(0, len(items), batch_size):
        batch = items[start:start + batch_size]

        for item in batch:
            results.append(call_api_with_retry(item, session))

        if start + batch_size < len(items):
            time.sleep(wait_seconds)

    return results
```

Code Walkthrough

- `BATCH_SIZE = 10` mirrors the `Split In Batches` node's batch size setting — the number of items sent before pausing.
- `WAIT_SECONDS = 2` mirrors the `Wait` node's delay — the pacing control that keeps the average request rate under the API's ceiling.
- `call_api_with_retry` is the Python equivalent of enabling `Retry On Fail` on the API-call node: it retries a failed request a bounded number of times, with a real delay between attempts (never zero), exactly as the video stresses.
- `process_in_batches` is the loop that replaces the Split-In-Batches-to-Wait-back-to-Split-In-Batches cycle: slice the list into chunks, fully process one chunk, sleep, move to the next chunk.
- The `if start + batch_size < len(items)` guard avoids an unnecessary final sleep after the last batch — a small but real detail, since waiting after the work is done wastes time for no benefit.

Dry Run

Take 25 items with `BATCH_SIZE = 10` and `WAIT_SECONDS = 2`:

1. `start = 0`: batch = items[0:10] (10 items). All 10 processed via `call_api_with_retry`. `start + batch_size (10) < 25` → sleep 2s.
2. `start = 10`: batch = items[10:20] (10 items). All processed. `20 < 25` → sleep 2s.
3. `start = 20`: batch = items[20:25] (5 items, the final partial batch). All processed. `30 < 25` is false → no sleep.
4. Loop ends. 25 items processed in 3 batches with 2 pauses, instead of 25 unthrottled calls fired back-to-back.

If item 15 times out on its first attempt, `call_api_with_retry` catches the exception, sleeps `RETRY_DELAY_SECONDS`, and retries — up to `MAX_RETRIES` times — before giving up. The rest of batch 2 and all of batch 3 are unaffected either way.

Complexity Analysis

- **Time:** O(n) API calls total, same as the brute-force version — batching doesn't change *how many* calls happen, only *how fast*. The added wall-clock time is `(n / batch_size) * wait_seconds`, which is the deliberate cost of staying under the rate limit.
- **Space:** O(batch_size) held in memory per batch, plus O(n) for the accumulated `results` list — no different from processing the full list at once, since nothing is persisted to disk or an external queue.
- These bounds are correct because batching and waiting are purely about *scheduling* the existing n calls, not about doing additional work per item.

Alternative Solutions

Exponential backoff with jitter is the classic alternative, and it's the wrong tool here. It's designed for *many independent clients* retrying against a shared resource, where jitter prevents them from retrying in sync and re-triggering the same spike. A single serial execution has no concurrency to desynchronize — applying backoff-with-jitter here adds complexity without solving a problem that exists.

A dedicated queue or message broker (e.g., a job queue with rate-limited workers) is appropriate when multiple independent processes need to share one API's rate budget. For a single workflow run, it's substantial infrastructure for a problem two nodes already solve.

Adaptive pacing, where the wait time is read dynamically from the API's `Retry-After` or custom rate-limit headers rather than hardcoded, is a genuine improvement worth adopting once you're past the "stop the 429s" stage — it turns a guess into a measured value.

Edge Cases

- **Empty item list:** the loop body never executes; no batches, no waits, no calls. Should be a no-op, not an error.
- **Fewer items than one batch size:** a single partial batch runs, no wait is needed afterward — handled by the `start + batch_size < len(items)` check.
- **Every item in a batch fails:** without `Retry On Fail` / `call_api_with_retry`, this would silently or loudly kill the run; with retries, each failing item gets bounded extra attempts before truly failing.
- **API returns a `Retry-After` value longer than your hardcoded wait:** respect the header, not your constant — ignoring it just produces the same 429 again.
- **Rate limit changes over time** (API tightens or loosens its ceiling): a static batch size and wait will eventually be wrong in one direction; this is a maintenance edge case, not a bug, but worth monitoring.

Common Mistakes

1. **Forgetting the loop-back connection** from `Wait` to `Split In Batches` — the workflow runs once, pauses once, then fires every remaining batch unthrottled.
2. **Setting the retry delay to zero** — the retry fires instantly into the same overloaded or glitching server and fails again for the same reason.
3. **Reaching for exponential backoff with jitter** on a single serial execution — unnecessary complexity for a problem that doesn't involve concurrent callers.
4. **Testing only on a small sample** (5–10 items) and assuming correctness — the rate ceiling is never crossed at that scale, so the bug is invisible until production volume.
5. **Hardcoding a wait time and never revisiting it** against the API's actual documented limit or `Retry-After` header — guessing instead of measuring.
6. **Treating every failure as a rate-limit failure** — a timeout in batch 30 can be a plain network blip, unrelated to pacing, and needs its own retry handling, not a bigger wait.

Interview Questions

- How would you design a system that calls a third-party API with a strict rate limit, from a single process?
- What's the difference between client-side throttling and server-side rate limiting, and why do you need to respect the server's limit even if your client is "fast enough"?
- When is exponential backoff with jitter the right tool, and when is it overkill?
- How would you make the wait time adaptive instead of a fixed constant?
- How do you distinguish a rate-limit failure from a transient network failure in your retry logic, and should they be retried the same way?

Similar Problems

- **LeetCode 1114 (Print in Order)** — synchronizing execution order across concurrent units, conceptually related to pacing and sequencing.
- **LeetCode 1116 (Print Zero Even Odd)** — another ordering/synchronization problem that builds intuition for controlled, turn-based execution, similar to batch-then-wait.
- **Designing a rate limiter (common system design question)** — token bucket / leaky bucket algorithms are the "bigger" version of the same idea this workflow solves with two nodes.
- **Producer-consumer problems** — relevant once you outgrow a single serial execution and need multiple workers sharing one rate budget.

Key Takeaways

- A 429 error means the server is defending a rate limit your client doesn't know about — the fix is to make the client respect it, not to retry harder.
- Batching plus a looped wait is enough to pace a single serial execution; you don't need a queue or broker for this shape of problem.
- The loop-back connection between `Wait` and `Split In Batches` is the entire mechanism — without it, you only throttle once.
- Exponential backoff with jitter solves a different problem (many concurrent clients), not this one.
- `Retry On Fail` with a non-zero delay handles the *other* kind of failure — transient errors unrelated to rate limiting — that pacing alone won't fix.
- Match the complexity of your fix to the actual shape of the problem. That's the engineering judgment call, not the node count.

Watch the Video

This entire pattern is built live, batch by batch, with the 429 wall shown failing and then fixed in real time. Watch it here: {LINK}

About the Series

This lesson is part of **Fun with Learning Technology**, a series that walks through practical automation, backend, and software engineering problems the way you'd actually encounter them at work — starting from a real failure, not a textbook definition, and building the fix piece by piece until it's production-solid.

Call To Action

If this saved you from debugging your own 429 wall at 2am, like the video on YouTube and subscribe — it's how the channel decides what to cover next. Subscribe to the Substack for the written breakdowns like this one, and drop a comment with the API or workflow you're currently fighting with; comments genuinely shape what gets built next. Share this with anyone on your team who's about to run their first big batch job.

Quick Review

Why does n8n hit a 429 wall on bulk API calls?

It fires requests as fast as the loop allows, ignoring the API's rate limit — no built-in pacing.

Which two built-in nodes fix API rate limiting?

Split In Batches (chunk items) + Wait (pause between chunks) — batch size and delay match the API's limit.

Split In Batches vs sending all items at once?

Split In Batches loops a fixed-size subset through the workflow, resetting state each pass — avoids bursting past the limit.

Common bug: workflow still gets banned despite batching?

No retry/backoff on individual failed requests — one bad item throws the whole batch instead of retrying with delay.

How to stop guessing the Wait duration?

Read the API's rate-limit headers (e.g. Retry-After) and set Wait dynamically instead of a hardcoded fixed delay.

Interview: how do Retry On Fail + Wait combine for resilience?

Retry On Fail retries a single failed node with backoff; Wait throttles pace between batches — together they survive transient errors without bans.
